
Introducción al análisis de series temporales

PID_00270813

Jordi Casas Roma

Tiempo mínimo de dedicación recomendado: 4 horas



Jordi Casas Roma

Licenciado en Ingeniería Informática por la Universidad Autónoma de Barcelona (UAB), máster de Inteligencia Artificial Avanzada por la Universidad Nacional de Educación a Distancia (UNED) y doctor en Informática por la UAB. Desde 2009 es profesor de los Estudios de Informática, Multimedia y Telecomunicación de la Universitat Oberta de Catalunya (UOC) y profesor asociado de la UAB. Es director del máster universitario de Ciencia de Datos de la UOC. Sus actividades docentes se centran en la minería de datos, el aprendizaje automático (*machine learning*) y el análisis de redes o grafos (*graph mining*). Desde 2010 pertenece al grupo de investigación KISON (K-ryptography and Information Security for Open Networks). Sus intereses de investigación incluyen temas relacionados con la privacidad, el aprendizaje automático y la minería de grafos.

Primera edición: febrero 2020

© Jordi Casas Roma

Todos los derechos reservados

© de esta edición, FUOC, 2020

Av. Tibidabo, 39-43, 08035 Barcelona

Realización editorial: FUOC

Ninguna parte de esta publicación, incluido el diseño general y la cubierta, puede ser copiada, reproducida, almacenada o transmitida de ninguna forma, ni por ningún medio, sea este eléctrico, químico, mecánico, óptico, grabación, fotocopia, o cualquier otro, sin la previa autorización escrita de los titulares de los derechos.

Índice

Introducción	5
Objetivos	6
1. Introducción a las series temporales	7
1.1. Conceptos preliminares	7
1.1.1. Datos transversales	7
1.1.2. Series temporales	9
1.1.3. Datos de panel.....	9
1.2. Definición y ejemplos.....	11
1.3. Objetivos y tareas analíticas.....	12
2. Componentes de una serie temporal	14
2.1. Tendencia general.....	14
2.2. Variaciones estacionales.....	15
2.3. Variaciones cíclicas.....	16
2.4. Variaciones aleatorias o irregulares	16
2.5. Descomposición de una serie temporal	17
3. Estacionariedad en series temporales	18
3.1. Definición de estacionariedad	18
3.2. Test de Dickey-Fuller aumentado.....	19
3.3. Convertir una serie temporal en estacionaria	20
4. Aprendizaje automático	22
4.1. Aprendizaje supervisado para series temporales	22
4.1.1. Predicciones en un paso o en múltiples pasos	23
4.1.2. Método de la ventana deslizante	23
4.1.3. Estrategias de predicción para múltiples pasos.....	26
4.2. Métricas para evaluación de series temporales	28
5. Ejemplos prácticos	31
5.1. Ejemplo 1: análisis de la concentración de CO ₂	31
5.1.1. Carga de los datos	31
5.1.2. Tendencia general	32
5.1.3. Movimientos estacionales	34
5.1.4. Estacionariedad de la serie temporal.....	37
5.2. Ejemplo 2: predicción del nivel de un embalse.....	38
5.2.1. Adquisición de datos	38
5.2.2. Análisi preliminar	39
5.2.3. Preparación de los datos	40

5.2.4.	Creación del modelo predictivo	41
5.2.5.	Resultados	42
Resumen		44
Glosario		45
Bibliografía		46

Introducción

En este módulo veremos una introducción al análisis de series temporales. Las series temporales son un tipo específico de datos que se aplican en muchos contextos actuales, tales como el análisis y la predicción de datos económicos y de consumo, de generación y consumo de energía, etc. En general, cualquier conjunto de datos en los que el tiempo sea la variable principal que rige la captura y comprensión de los datos puede ser tratado y analizado como una serie temporal.

Iniciaremos este módulo con una introducción al concepto de *series temporales*, definiendo qué es una serie temporal, cuáles son las principales características, objetivos y tareas analíticas asociados a este tipo de datos. Veremos los cuatro componentes principales que forman una serie temporal: variaciones a largo plazo o tendencia general; variaciones a corto plazo, que pueden ser estacionales o cíclicas, y, finalmente, variaciones aleatorias o irregulares. A continuación, presentaremos el concepto de *estacionariedad* en las series temporales, discutiremos su importancia y veremos algunos métodos básicos para determinar si una serie es estacionaria y, en caso de no serlo, cómo se puede transformar en una serie estacionaria.

Seguiremos viendo cómo hay que aplicar los métodos estándares de aprendizaje automático (*machine learning*) en este tipo de datos, que dadas sus peculiaridades necesitarán ciertas transformaciones para poder entrenar un modelo predictivo de aprendizaje automático. En relación con los modelos predictivos, también revisaremos algunas de las métricas más empleadas para evaluar la bondad en la predicción de series temporales.

Finalmente, en el último capítulo veremos un ejemplo de cómo analizar y descomponer una serie temporal empleando el lenguaje Python y algunas de las librerías más relevantes para el análisis de series temporales y aprendizaje automático.

Objetivos

En los materiales didácticos de este módulo encontraremos las herramientas indispensables para asimilar los siguientes objetivos:

1. Entender qué es una serie temporal y conocer sus principales características.
2. Comprender cuáles son los componentes de una serie temporal y cómo podemos identificarlos.
3. Conocer los procesos de transformación de datos que nos permitirán reestructurar una serie temporal para poder entrenar un modelo de aprendizaje automático.
4. Entender el concepto de *estacionariedad* en series temporales y ser capaz de determinar si una serie temporal es estacionaria.
5. Conocer las principales métricas de evaluación de series temporales, con el fin de poder evaluar la bondad de un modelo predictivo.

1. Introducción a las series temporales

En los últimos años hemos sido testigos de la aplicación generalizada de estadísticas y aprendizaje automático para obtener información procesable y valor comercial de los datos en casi todos los sectores industriales. Los analistas y científicos de datos deben abordar diferentes tipos de problemas, escenarios, dominios y, también, diferentes tipos de conjuntos de datos.

Las series temporales son conjuntos de datos con ciertas características y particularidades que las hacen especialmente relevantes en muchas áreas de investigación y negocio, tales como el análisis del producto interior bruto, los volúmenes de ventas, los precios de las acciones, los atributos climáticos, la generación y el consumo de energía, etc.

1.1. Conceptos preliminares

Los analistas y científicos de datos se encuentran con muchos tipos diferentes de datos en sus proyectos de análisis. Comprender qué tipo de datos se necesitan para resolver un problema y qué tipo de datos se pueden obtener de las fuentes disponibles es importante para formular el problema y elegir la metodología correcta para el análisis.

Podemos establecer una tipología básica que incluye tres tipos primarios de datos, teniendo en cuenta la dimensión tiempo en la estructura interna de los mismos:

- datos transversales
- series temporales
- datos de panel

A continuación veremos, desde un enfoque práctico y con algunos ejemplos, cómo son y cuáles son las principales características de estos tipos primarios de datos.

1.1.1. Datos transversales

Los **datos transversales** (*cross-sectional data*, en inglés) de una población se obtienen tomando observaciones de múltiples individuos en el mismo instante.

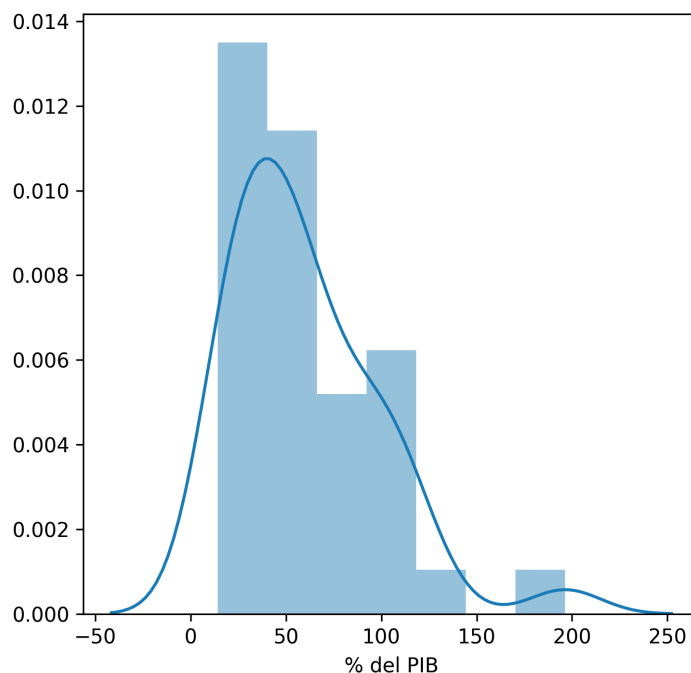
Alternativamente, también pueden comprender observaciones tomadas en diferentes momentos, aunque en estos casos se asume que el tiempo no tiene un papel significativo en el análisis de los datos.

En esencia, los datos transversales representan una instantánea en un momento concreto, aunque pueden obtenerse en un período considerable (meses e incluso años), siempre y cuando el tiempo no desempeñe un papel importante cuando se utilicen para fines analíticos.

Ejemplo de datos transversales

La figura 1 ilustra un ejemplo de datos transversales univariante de la deuda pública como porcentaje del producto interior bruto (PIB) de ochenta y cinco países en el año 2016. Al tomar los datos de un solo año, aseguramos su naturaleza transversal. La figura combina un histograma normalizado y un gráfico de densidad del núcleo para resaltar diferentes propiedades estadísticas de los datos.

Figura 1. Deuda pública como porcentaje del PIB de ochenta y cinco países en el año 2016



A partir de la figura se puede deducir que la mayoría de países acumulan una deuda pública de entre el 25 % y el 50 % de su PIB, aunque en un buen número de países asciende alrededor del 100 %. También podemos observar un pico menor cerca del 200 % del PIB.

Este tipo de datos es muy empleado en tareas de minería de datos y aprendizaje automático (*machine learning*, en inglés). La mayoría de algoritmos supervisados y no supervisados han sido desarrollados explícitamente para este tipo de datos.

1.1.2. Series temporales

En contraposición a los datos transversales, las **series temporales** (*time series*, en inglés) se componen de observaciones cuantitativas sobre una o más características medibles de una entidad individual y tomadas en múltiples puntos en el tiempo.

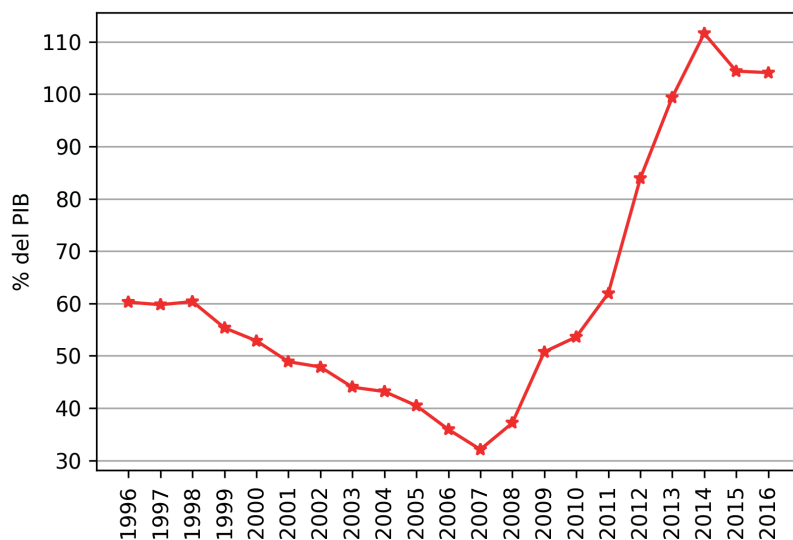
Como veremos en los apartados posteriores con más profundidad, las series temporales se caracterizan, típicamente, por varias estructuras internas, como la tendencia, la estacionalidad, la estacionariedad, la autocorrelación, etc. Las estructuras internas de los datos de series temporales requieren una formulación y unas técnicas especiales para su análisis.

Ejemplo de serie temporal

Un ejemplo clásico de datos de series temporales son los datos económicos, ya sean de empresas, entidades o países. En este caso, veremos un ejemplo que incluye datos referentes a la deuda del gobierno de España durante el período entre 1996 y 2016.

Entre otros datos, se proporcionan los indicadores de desarrollo mundial que incluyen multitud de datos anuales agregados por país o región, tales como la deuda pública o el gasto militar.

Figura 2. Deuda del gobierno de España entre los años 1996 y 2016



Estos datos son un claro ejemplo de serie temporal, en la que vemos cómo evoluciona una determinada variable (en este caso, la deuda nacional) en función de la variable tiempo. En este caso concreto, los datos tienen una periodicidad anual, muy típica en los datos económicos.

1.1.3. Datos de panel

Hasta ahora, hemos visto datos tomados de múltiples individuos o instancias en un punto temporal –o sin tener en cuenta la variable tiempo– (datos trans-

WDI

Los indicadores del desarrollo mundial (*world development indicators*, WDI) constituyen la principal compilación de estadísticas internacionales sobre desarrollo global del Banco Mundial. Utilizando fuentes reconocidas oficialmente e incluyendo estimaciones nacionales, regionales y mundiales, el WDI proporciona acceso a, aproximadamente, mil seiscientos indicadores para doscientas diecisiete economías, con algunas series temporales que se extienden más de cincuenta años.

Enlace de interés

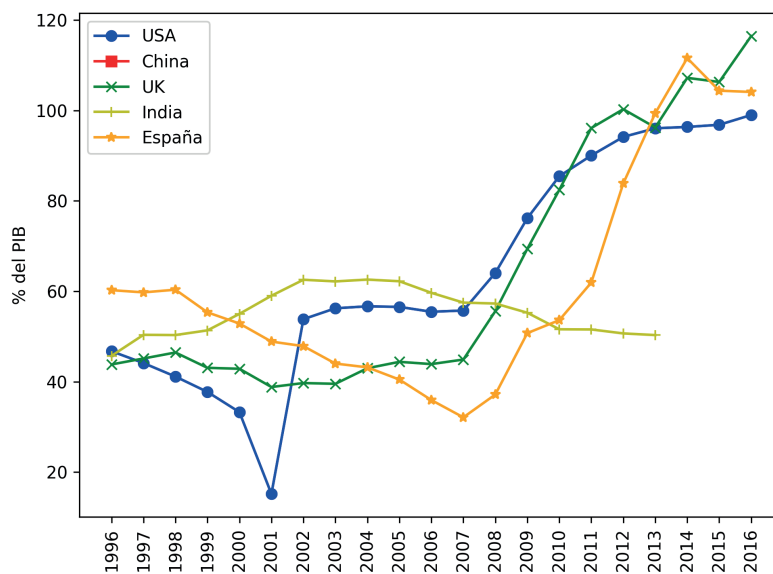
Los datos empleados son de acceso público y se pueden consultar en la página web del Banco Mundial. Además, se pueden descargar en: <https://datacatalog.worldbank.org/dataset/world-development-indicators>.

versales) o tomados de una entidad individual pero en múltiples puntos en el tiempo (series temporales). Sin embargo, si observamos múltiples entidades en múltiples puntos temporales, obtenemos los **datos de panel** (*panel data*, en inglés), también conocidos como **datos longitudinales**.

Ejemplo de datos de panel

Extendiendo nuestro ejemplo anterior sobre la deuda pública, consideremos ahora cinco países durante el período entre 1996 y 2016. Los datos resultantes serán un conjunto de datos del panel. La figura 3 ilustra los datos del panel en este escenario.

Figura 3. Deuda de cinco países entre los años 1996 y 2016



Según la relación entre individuos y tiempo, podemos clasificar los paneles de datos en dos grupos:

- Un **panel corto** es aquel en el que el número de individuos es menor al tiempo.
- Un **panel largo** es aquel en el que hay más individuos que períodos de tiempo.

Un modelo de regresión común para el análisis de datos de panel tiene esta forma:

$$y_{it} = a_i + bx_{it} + \epsilon_{it}, \quad (1)$$

donde y es la variable dependiente, x es la variable independiente, a y b son coeficientes, i y t son los índices para los individuos y el tiempo y, finalmente, ϵ_{it} es el error.

1.2. Definición y ejemplos

Como hemos visto en la sección anterior, las series temporales son un tipo de datos que miden cómo cambian las cosas con el tiempo. En las series temporales, el tiempo deviene un eje primario que proporciona una fuente de información esencial para entender y procesar los datos y que proporciona la disposición cronológica de los mismos.

Los datos de una serie temporal forman una secuencia de observaciones cuantitativas sobre un sistema o proceso en puntos sucesivos en el tiempo.

Generalmente, los conjuntos de datos de series temporales tienen tres aspectos en común:

- El tiempo es un eje primario.
- Los datos generalmente se recopilan por orden temporal.
- Los puntos en el tiempo son equiespaciados, es decir, mantienen la misma distancia entre ellos.

Existen multitud de ejemplos de datos de series temporales. Veamos algunos ejemplos::

- Datos relacionados con factores económicos, como el PIB, los volúmenes de ventas o el precio de unas determinadas acciones.
- Datos climatológicos, que pueden incluir la temperatura, las precipitaciones, la humedad, etc.
- Datos de consumo en redes eléctricas o volumen de datos en redes de telecomunicaciones.

En realidad, cualquier dato que se pueda registrar o medir con cierta periodicidad (años, meses, días, horas, minutos, segundos, etc.) podría constituir una serie temporal. La frecuencia de observación depende de la naturaleza del dato. Por ejemplo, el PIB, que se utiliza para medir el progreso económico anual de un país, se publica anualmente. En cambio, la información sobre el precio de las acciones o ciertos datos climáticos están disponibles cada segundo. En general, cuando los datos son tomados de forma autónoma por sensores automáticos, como en el caso de las redes de comunicaciones o IoT, se generan datos de series temporales con periodicidades que puedan llegar a varios órdenes de magnitud por debajo del segundo (centésimas, milésimas, etc.).

IoT

El concepto *internet de las cosas* (*Internet of Things*, IoT) se refiere a la interconexión digital de objetos cotidianos con internet.

Las series temporales pueden representarse como un conjunto ordenado de tuplas (t_i, obs_i) , donde t_i es la marca de tiempo i y obs_i es el valor de la variable cuantitativa en el momento i . Adicionalmente, también podemos representar y almacenar una serie temporal como una tabla, con dos columnas como mínimo, donde la primera columna es la marca de tiempo y la segunda es el valor de la variable observada en este instante de tiempo. La tabla 1 muestra un ejemplo de cómo se representa y almacena la información de una serie temporal, donde cada fila contiene la información sobre la marca de tiempo y el valor de la variable observada.

Tabla 1. Ejemplo de tabla de datos de una serie temporal.

Tiempo	Observación
1	35
2	42
3	41
4	47
5	45
...	...

Cuando una serie temporal tiene una única variable observada, decimos que es **univariable**. Por el contrario, cuando una serie temporal presenta la forma $(t_i, obs_{i,1}, obs_{i,2}, \dots, obs_{i,m})$ se le llama **serie temporal multivariante** (en este caso concreto con m observaciones en cada instante de tiempo). La tabla 2 muestra un ejemplo de serie temporal multivariante con m variables u observaciones.

Como ejemplo de serie temporal univariante podemos pensar en datos que registren el consumo de una red eléctrica cada minuto o hora. Un ejemplo de caso de una serie temporal multivariante podría ser un conjunto de datos climatológicos, que agrupan varias observaciones en cada instante de tiempo (tales como la temperatura, la humedad, la presión del aire y la velocidad del viento).

Tabla 2. Ejemplo de tabla de datos de una serie temporal

Tiempo	Obs_1	Obs_2	...	Obs_m
1	35	0.3	...	875.56
2	42	0.5	...	657.82
3	41	0.6	...	756.56
4	47	0.7	...	813.98
5	45	0.8	...	918.01
...	...	...	...	...

1.3. Objetivos y tareas analíticas

En relación con los datos de series temporales, y de forma similar a otros procesos de minería de datos o aprendizaje automático, distinguimos dos tareas principales, aunque con métodos y objetivos claramente diferenciados:

- El **análisis de series temporales** (*time series analysis*, TSA) tiene como principal objetivo comprender e interpretar las fuerzas subyacentes que produ-

cen el estado observado de un sistema o proceso a lo largo del tiempo. El TSA implica el desarrollo de modelos que describan una serie temporal para comprender las causas subyacentes, en muchos casos empleando herramientas estadísticas clásicas. Esta tarea busca el porqué de un conjunto de datos e intenta explicar la estructura interna subyacente que pueda haber en los puntos de datos tomados a lo largo del tiempo (como la autocorrelación, la tendencia o la variación estacional). Esto a menudo implica hacer suposiciones sobre la forma de los datos y descomponer la serie temporal.

- La **predicción de series temporales** (*time series forecasting*, TSF) intenta predecir observaciones futuras con base en los datos históricos contenidos en la serie temporal. Su objetivo principal es crear modelos que capturen los patrones subyacentes a los datos y permitan pronosticar el estado futuro del sistema o proceso en términos de características observables.

2. Componentes de una serie temporal

En este apartado nos centraremos en definir las características particulares de las series temporales que requieren un tratamiento matemático especial. En general, una serie temporal puede ser definida como la suma de cuatro factores:

$$\text{Componentes} \left\{ \begin{array}{l} \text{Tendencia general (general trend)} \\ \text{Movimientos a corto plazo} \left\{ \begin{array}{l} \text{Estacionales (seasonal variations)} \\ \text{Cíclicos (cyclical variations)} \end{array} \right. \\ \text{Variaciones aleatorias o irregulares (random or irregular variations)} \end{array} \right.$$

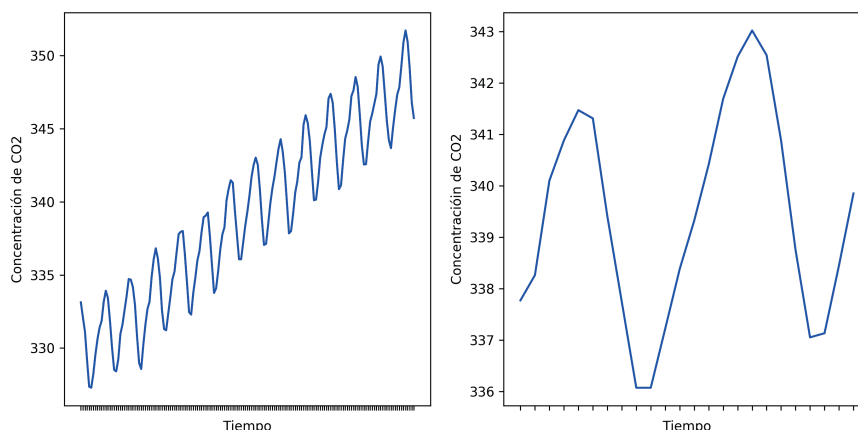
2.1. Tendencia general

La **tendencia general**, también conocida como **movimiento a largo plazo**, se refiere al comportamiento general de los datos de series temporales a aumentar o disminuir durante un largo período de tiempo. Aunque podemos observar cómo las tendencias cambian de dirección múltiples veces en diferentes momentos del período de tiempo que estamos observando, suele existir una tendencia general única para todo el conjunto de datos, que indicará si los valores, de forma global en toda la serie temporal, tienen un comportamiento al alza, a la baja o estable.

Ejemplo de tendencia general

Supongamos que disponemos de una serie temporal con información sobre la concentración de CO₂ que se encuentra en el aire de un determinado país o región durante un periodo considerable de tiempo. La figura 4 ilustra los datos de este escenario.

Figura 4. Ejemplo de tendencia general. Sobre los mismos datos, en la figura de la izquierda vemos un período de tiempo mucho más largo que en la figura de la derecha.



Podemos ver que si observamos el comportamiento general de esta variable a largo plazo (figura de la izquierda), hay una clara tendencia al alza (aumento) en la concentración de CO₂ en el aire. En el caso de no disponer de un conjunto suficientemente grande de

datos, es posible que solo podamos ver una ventana demasiado pequeña de los datos, que estos no nos permitan identificar el comportamiento general y que, por lo tanto, no podamos ver la tendencia general. Por ejemplo, la figura de la derecha muestra los mismos datos con una ventana de tiempo menor, en la que no es posible determinar el tipo de movimiento a largo plazo.

2.2. Variaciones estacionales

Las **variaciones estacionales**, que forman parte de los **movimientos a corto plazo**, son patrones rítmicos que se observan de manera regular y periódica (generalmente con una periodicidad anual). Es decir, las variaciones estacionales presentan el mismo patrón (o similar) durante un período de doce meses. Lógicamente, esta componente solo estará presente en la serie temporal si los datos tienen una frecuencia suficientemente pequeña, como por ejemplo de horas, días o semanas. Estas variaciones se deben, principalmente, a dos factores distintos:

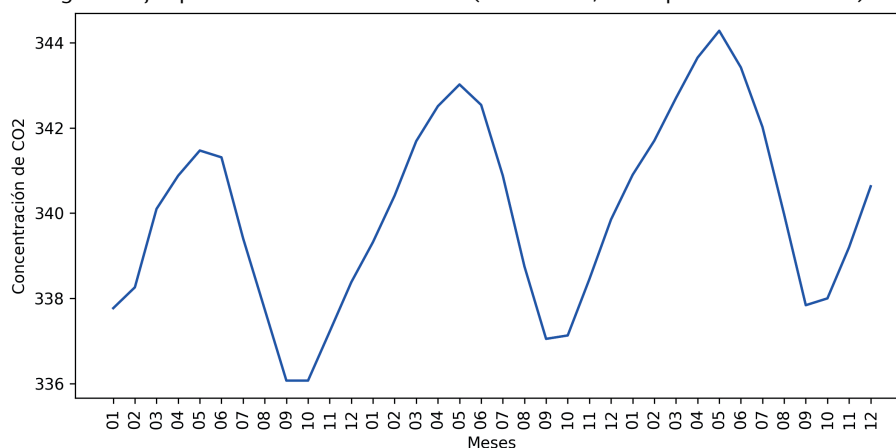
- Las diferentes **estaciones** o **condiciones climáticas** tienen un papel importante en las variaciones estacionales. Por ejemplo, el volumen de precipitaciones o la temperatura tienen efectos claros sobre la producción de cultivos o la venta de determinados productos (como aires acondicionados). Es decir, con una periodicidad anual, las condiciones naturales de nuestro entorno tienen un efecto importantísimo sobre nuestra actividad y nuestro mundo.
- Las convenciones establecidas por el ser humano tienen claros efectos sobre nuestros patrones de movimiento, consumo, etc. Son claros ejemplos de este componente a corto plazo los períodos vacacionales, el calendario laboral, etc.

En general, ambos factores tienen una influencia muy importante sobre la repetitividad y la periodicidad de cualquier actividad natural y humana.

Ejemplo de variaciones estacionales

Siguiendo con el ejemplo anterior sobre la concentración de CO_2 , la figura 5 muestra que durante la primavera los niveles de concentración de CO_2 aumentan hasta alcanzar los máximos anuales. Por contra, durante el otoño, y en especial durante los meses de septiembre y octubre, los niveles de contaminación del aire se reducen a los mínimos anuales.

Figura 5. Ejemplo de variaciones estacionales (en este caso, de un período de tres años)



Aunque la tendencia general de la serie es aumentar, podemos ver que este patrón se repite anualmente.

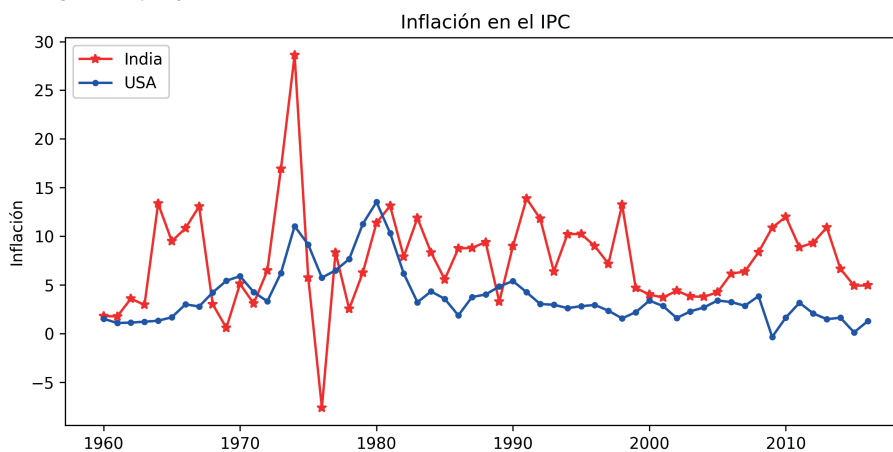
2.3. Variaciones cíclicas

Las **variaciones cíclicas**, que también forman parte de los **movimientos a corto plazo**, son oscilaciones en una serie de tiempo que operan con una periodicidad superior a la anual. Un período completo es un ciclo, pero un ciclo no tiene un período de tiempo fijo específico.

Ejemplo de variaciones cíclicas

En el análisis de datos económicos o de negocios aparece, generalmente, un movimiento cíclico que responde a los períodos de prosperidad, recesión, depresión y recuperación que periódicamente aparecen en la mayoría de economías nacionales o globales. Las subidas y bajadas en los negocios dependen de la naturaleza conjunta de las fuerzas económicas y de la interacción entre ellas.

Figura 6. Ejemplo de variaciones cíclicas



La figura 6 muestra los cambios cíclicos que ocurren en la inflación del índice de precios al consumidor (IPC) para la India y los Estados Unidos durante el período de 1960 a 2016. Ambos países exhiben patrones cíclicos en la inflación del IPC aproximadamente cada dos o tres años. La periodicidad de estos ciclos no está fijada y depende de múltiples y complejas variables económicas, tanto a nivel local como global.

2.4. Variaciones aleatorias o irregulares

Finalmente, las **variaciones aleatorias o irregulares** son el último factor que causa fluctuaciones en los datos de series temporales. Estas fluctuaciones son imprevistas, incontrolables, impredecibles y erráticas (los terremotos, las guerras, las inundaciones, las hambrunas y cualquier otro desastre natural).

Las variaciones aleatorias o irregulares son estocásticas y no pueden enmarcarse en un modelo matemático para una predicción futura. Este tipo de error se debe a la falta de información sobre variables explicativas que pueden modelar estas variaciones o debido a la presencia de un ruido aleatorio en los datos.

2.5. Descomposición de una serie temporal

Estos cuatro componentes se combinan para generar las series temporales. Se pueden hacer suposiciones sobre estos componentes tanto en el comportamiento como en la forma en la que se combinan, lo que les permite modelarse utilizando métodos estadísticos tradicionales. Estos componentes también pueden ser la forma más efectiva de hacer predicciones sobre valores futuros.

En este sentido, una serie temporal puede expresarse de forma matemática:

$$x_t = f_t + s_t + c_t + e_t, \quad (2)$$

donde f , s , c y e representan, respectivamente, los componentes de tendencia, estacionalidad, variaciones cíclicas y variaciones aleatorias o irregulares. En la fórmula anterior, t es el índice de tiempo de la serie, que toma valores sucesivos e igualmente espaciados en el tiempo; es decir, $t = 1, 2, 3, \dots, N$.

El objetivo del análisis de series temporales es descomponer una serie temporal en sus características constitutivas y desarrollar modelos matemáticos para cada una. Estos modelos se utilizan para comprender qué causa el comportamiento observado de la serie temporal y para predecir la serie para futuros puntos en el tiempo.

3. Estacionariedad en series temporales

En este apartado discutiremos el concepto de *estacionariedad* de las series temporales (*stationarity*, en inglés). Esta propiedad es importante en el análisis de series temporales, ya que muchas técnicas de análisis estadístico parten de la suposición de que el conjunto de datos que se está analizando es estacionario.

3.1. Definición de estacionariedad

Una serie temporal es **estacionaria** si las propiedades de la serie temporal (por ejemplo, media, varianza, autocorrelación, etc.) no cambian con el tiempo.

La mayoría de los métodos de pronóstico estadístico se basan en el supuesto de que las series de tiempo pueden hacerse aproximadamente estacionarias mediante el uso de transformaciones matemáticas. La estacionariedad es un concepto especialmente relevante en las series temporales, ya que muchas teorías econométricas clásicas se derivan de los supuestos de estacionariedad.

A continuación enumeramos las dos razones principales que justifican la importancia del concepto de *estacionariedad* de las series temporales:

- 1) Una serie temporal estacionaria es relativamente fácil de predecir, ya que sus propiedades estadísticas se mantienen constantes a lo largo del tiempo.
- 2) Otra razón para tratar de estacionarizar una serie de tiempo es poder obtener estadísticas de muestra significativas, como medias, variaciones y correlaciones con otras variables, que se pueden usar para obtener más información y comprender mejor sus datos.

Hay dos formas importantes de estacionariedad:

- **Estacionariedad fuerte** (*strongly stationary*). Se dice que una serie temporal es fuertemente estacionaria cuando todas sus propiedades son invariables por el cambio del origen del tiempo o la traducción del tiempo.

- **Estacionariedad débil** (*weakly stationary*). Se dice que una serie temporal es estacionaria de segundo orden, o débilmente estacionaria, cuando sus funciones de media y autocovarianza son invariantes por el cambio del origen del tiempo o la traducción del tiempo.

En realidad, la mayoría de las series de tiempo económicas y comerciales están lejos de ser estacionarias cuando se expresan en sus unidades de medida originales, e incluso después del ajuste estacional, típicamente exhibirán tendencias, ciclos, aleatoriedad y otros comportamientos no estacionarios.

3.2. Test de Dickey-Fuller aumentado

El test de Dickey-Fuller aumentado (*augmented Dickey-Fuller*, ADF) está diseñado para indicar explícitamente si una serie temporal univariante es estacionaria. Esta prueba se considera una prueba de raíz unitaria para estacionariedad. Una raíz unitaria (también llamada *proceso de raíz unitaria* o *proceso estacionario de diferencia*) es una tendencia estocástica en una serie temporal. Si una serie temporal tiene una raíz unitaria, muestra un patrón sistemático que es impredecible.

El test de Dickey-Fuller aumentado utiliza un modelo autorregresivo y optimiza un criterio de información en múltiples valores de retraso diferentes. La hipótesis nula de la prueba es que la serie de tiempo puede ser representada por una raíz unitaria, que no es estacionaria (tiene alguna estructura dependiente del tiempo). La hipótesis alternativa (que rechaza la hipótesis nula) es que la serie temporal es estacionaria.

- **Hipótesis nula** (H_0). Si no se rechaza, sugiere que la serie temporal tiene una raíz unitaria, lo que significa que **no es estacionaria**. Tiene alguna estructura dependiente del tiempo.
- **Hipótesis alternativa** (H_1). Se rechaza la hipótesis nula; sugiere que la serie temporal no tiene una raíz unitaria, lo que significa que **es estacionaria**. No tiene una estructura dependiente del tiempo.

Interpretamos este resultado utilizando el valor p de la prueba. Un valor p por debajo de un umbral (como 5 % o 1 %) sugiere que rechazemos la hipótesis nula (estacionaria); de lo contrario, un valor p por encima del umbral sugiere que no rechazemos la hipótesis nula (no estacionaria):

- Valor $p > 0,05$: no se puede rechazar la hipótesis nula (H_0), los datos tienen una raíz unitaria y son **no estacionarios**.
- Valor $p \leq 0,05$: rechaza la hipótesis nula (H_0), los datos no tienen una raíz unitaria y son **estacionarios**.

Ejemplo de test de Dickey-Fuller aumentado

Supongamos que realizamos el test sobre los datos de una serie temporal, con el resultado que se incluye a continuación:

```
ADF Statistic: -6.781424
p-value: 0.000000
Critical Values:
5\%: -2.862
10\%: -2.567
1\%: -3.430
```

El valor estadístico de prueba es de -6.7 . Cuanto más negativo sea este valor, más probable es que rechacemos la hipótesis nula (tenemos un conjunto de datos estacionario).

Como parte de la salida, obtenemos una tabla de búsqueda para ayudar a determinar la estadística ADF. Podemos ver que nuestro valor estadístico de -6.7 es menor que el valor de -3.430 al 1 %.

Esto sugiere que podemos rechazar la hipótesis nula con un nivel de significación de menos del 1 % (una baja probabilidad de que el resultado sea una casualidad estadística). Rechazar la hipótesis nula significa que el proceso no tiene raíz unitaria y, a su vez, que la serie temporal **es estacionaria** o no tiene una estructura dependiente del tiempo.

3.3. Convertir una serie temporal en estacionaria

Los conjuntos de datos de series temporales pueden contener tendencias y estacionalidad, que pueden eliminarse antes del modelado. Existen diferentes técnicas para convertir una serie temporal no estacionaria en una serie temporal estacionaria. Dichas técnicas, en esencia, se basan en identificar y eliminar tendencias y efectos estacionarios de una serie temporal.

Las **tendencias** pueden dar lugar a una **media variable** a lo largo del tiempo, mientras que la **estacionalidad** puede dar lugar a una **varianza variable** a lo largo del tiempo, lo que define una serie temporal como no estacionaria. Si la serie temporal no es estacionaria, a menudo podemos transformarla en estacionaria utilizando una técnica popular llamada **transformación por diferencia** (*difference transform*, en inglés).

La diferenciación es una transformación de datos popular y ampliamente utilizada para hacer estacionarias las series temporales. Se realiza restando la observación anterior de la observación actual, tal y como se indica en la siguiente expresión:

$$\text{diferencia}(t) = \text{obs}(t) - \text{obs}(t - 1), \quad (3)$$

donde $\text{obs}(t)$ indica el valor de la observación en el instante de tiempo t .

En el caso de series temporales, debemos tomar la diferencia entre observaciones consecutivas. Esta operación se llama **diferencia de retraso-1** (*lag-1 difference*). La diferencia de retraso se puede ajustar para adaptarse a la estructura temporal específica de cada conjunto de datos. Para series temporales con un componente estacional, se puede esperar que el retraso sea el período (ancho) de la estacionalidad.

Algunas estructuras temporales en una serie temporal aún pueden existir después de realizar una operación de diferenciación, como en el caso de una tendencia no lineal. Como tal, el proceso de diferenciación puede repetirse más de una vez hasta que se haya eliminado toda la dependencia temporal. El número de veces que se realiza la diferenciación se denomina **orden de diferencia** (*difference order*).

La diferencia se puede utilizar para transformar sus series temporales no estacionarias en series temporales estacionarias mediante estas acciones:

- Eliminar tendencias.
- Eliminar la estacionalidad.

4. Aprendizaje automático

Dadas las características propias de una serie temporal que hemos visto hasta ahora, debemos realizar algunos pasos preliminares de reestructuración de datos para poder aplicar los modelos estándares de aprendizaje automático en las series temporales.

En este apartado veremos estos pasos preliminares que nos permitirán entrenar los modelos de aprendizaje automático supervisados con los datos de una serie temporal y utilizarlos para predecir valores futuros de la serie temporal. Por lo tanto, en este apartado nos centraremos en la tarea de **predicción de series temporales** (TSF) que hemos descrito anteriormente. Además, nos centraremos en el caso del **aprendizaje supervisado**, es decir, emplearemos los datos históricos de la serie temporal para generar y entrenar el modelo, con el objetivo final de predecir, de la forma más ajustada posible, los valores futuros de las observaciones de la serie temporal.

Finalmente, en el último subapartado de este apartado revisaremos algunas de las métricas para la evaluación de modelos predictivos en series temporales.

4.1. Aprendizaje supervisado para series temporales

Generalmente, en los métodos supervisados partiremos de un conjunto de datos correctamente etiquetados, D , en el que distinguimos la siguiente estructura:

$$D_{n,m} = \begin{pmatrix} d_1 = & a_{1,1} & a_{1,2} & \cdots & a_{1,m} & c_1 \\ d_2 = & a_{2,1} & a_{2,2} & \cdots & a_{2,m} & c_2 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ d_n = & a_{n,1} & a_{n,2} & \cdots & a_{n,m} & c_n \end{pmatrix}$$

donde:

- n es número de elementos o instancias.
- m es el número de atributos del conjunto de datos o también su dimensionalidad.
- d_i indica cada una de las instancias del juego de datos.
- a_i indica cada uno de los atributos descriptivos.
- c_i indica el atributo objetivo o clase que hay que predecir para cada elemento.

Como podemos ver, esta estructura difiere sensiblemente de la estructura que presentan las series temporales. Por lo tanto, para poder aplicar los algoritmos estándares de aprendizaje automático a las series temporales, deberemos efectuar una serie de transformaciones que nos permitan reestructurar los datos en un formato similar al que acabamos de presentar.

4.1.1. Predicciones en un paso o en múltiples pasos

En la predicción de series temporales podemos estar interesados en crear predicciones a corto plazo (por ejemplo, en el siguiente instante de tiempo) o, por el contrario, podemos desear crear modelos que realicen predicciones a medio o largo plazo.

Según la necesidad de predicción del modelo, hablaremos de dos tipos básicos:

- **One-step forecast.** El modelo predice el siguiente instante de tiempo; es decir, $t + 1$.
- **Multi-step forecast.** El modelo predice valores para dos o más instantes de tiempo futuros.

Dependiendo del problema en cuestión, deberemos escoger entre la predicción de un paso o de varios pasos, pudiendo escoger en este segundo caso el número exacto de predicciones futuras que deseamos obtener del modelo. En general, y como es obvio, las predicciones a valores de futuro mayores suelen ser menos precisas.

4.1.2. Método de la ventana deslizante

El proceso para reestructurar una secuencia de datos de series temporales para que parezcan un problema de aprendizaje supervisado es bastante simple. La tabla 3 muestra un pequeño ejemplo de serie temporal, con cinco registros que contienen una información estándar de una serie temporal; es decir, tuplas en formato (t_i, obs_i) .

Tabla 3. Datos de una serie temporal univariante

Tiempo	Variable
1	35
2	42
3	41
4	47
5	45

Para reestructurar la información creamos una nueva tabla, donde cada fila i contiene como variables conocidas (X) los valores de la serie temporal en el instante $t = i - 1$, y, como valor objetivo o valor a predecir (y), el valor de la serie temporal en el instante de tiempo $t = i$.

La tabla 4 muestra el resultado de este proceso de reestructuración. Como podemos ver, cada fila contiene una tupla de valores (obs_x, obs_y) que indican el valor de entrada y de salida, respectivamente. De este modo, el modelo intentará aprender a generar la salida en función del valor de entrada. Es decir, podemos reestructurar este conjunto de datos de series temporales como un problema de aprendizaje supervisado mediante el uso del valor en el paso de tiempo anterior para predecir el valor en el siguiente paso de tiempo.

Tabla 4. Datos de una serie temporal univariante procesados para un modelo supervisado

x	y
35	42
42	41
41	47
47	45
45	?

Es importante notar que, aunque la tabla resultante tiene el mismo número de filas, la última fila de los datos reestructurados será el primer valor que el modelo deberá predecir, ya que en los datos originales no está disponible el valor de salida. Además, podemos ver que se preserva el orden entre las observaciones para entrenar un modelo supervisado. Una vez que un conjunto de datos de series de tiempo se prepara de esta manera, se puede aplicar cualquier algoritmo de aprendizaje automático estándar, siempre que se preserve el orden de las filas.

El uso de instantes de tiempo anteriores para predecir el siguiente instante de tiempo se denomina **método de la ventana deslizante** (*sliding window method*, en inglés), aunque también es conocido como **método de retraso** (*lag method*, en inglés). El número de pasos de tiempo anteriores se denomina **ancho de ventana** (*window width*) o tamaño del retraso (*size of the lag*). Esta ventana deslizante constituye la base de la conversión de cualquier conjunto de datos de series temporales a un problema de aprendizaje supervisado.

El enfoque de ventana deslizante puede usarse en una serie de tiempo que tiene más de un valor (es decir, series temporales multivariadas), que son conjuntos de datos donde se observan dos o más variables en cada momento. Generalmente, se escoge solo una de las variables como variable objetivo (*target*), que es la que el modelo intentará predecir. El procedimiento es similar, utilizando todas las observaciones del instante $t = i - 1$ como variables de entrada en la fila i , y la observación seleccionada del instante $t = i$ como variable de salida o variable objetivo en la fila i .

Tabla 5. Datos de una serie temporal multivariante

Tiempo	Variable 1	Variable 2
1	0.1	35
2	0.2	42
3	0.3	41
4	0.3	47
5	0.4	45

La tabla 5 muestra una serie temporal multivariante que contiene dos observaciones para cada instante de tiempo.

Tabla 6. Datos de una serie temporal multivariante procesados para un modelo supervisado, donde la variable de salida corresponde a la variable 1 de la tabla 5.

X_1	X_2	y_1
0.1	35	0.2
0.2	42	0.3
0.3	41	0.3
0.3	47	0.4
0.4	45	?

Tabla 7. Datos de una serie temporal multivariante procesados para un modelo supervisado, donde la variable de salida corresponde a la variable 2 de la tabla 5.

X_1	X_2	y_1
0.1	35	42
0.2	42	41
0.3	41	47
0.3	47	45
0.4	45	?

En el caso de escoger la **variable 1** como variable de salida o *target*, el resultado del proceso de reestructuración sería el de la tabla 6. Podemos ver que el conjunto de entrenamiento para el modelo contiene dos variables de entrada (X_1 y X_2) y una variable de salida (y_1). El ejemplo complementario, en el que se selecciona la variable 2 como variable de salida, se representa en la tabla 7.

Hasta este punto hemos visto cómo usar el método de la ventana deslizante para modelos basados en predicciones a un paso de distancia (*one-step forecast*). Pero este método también puede servir para realizar pronósticos en el caso de predicciones a dos o más pasos de distancia (*multi-step forecast*).

Para ejemplificar este proceso, consideremos el mismo conjunto de datos de series temporal univariantes de la tabla 3. Como se ilustra en la tabla 8, podemos enmarcar esta serie temporal como un conjunto de datos de pronóstico de dos pasos para el aprendizaje supervisado con un ancho de ventana igual a uno.

Tabla 8. Datos de una serie temporal univariante procesados para un modelo supervisado de pronóstico de dos pasos (*multi-step forecast*).

X_1	y_1	y_2
35	42	41
42	41	47
41	47	45
47	45	?
45	?	?

Podemos ver que las últimas dos filas no pueden usarse para entrenar un modelo supervisado. En este modelo hay que considerar la carga sobre las variables de entrada. Específicamente, un modelo supervisado solo tiene X_1 como dato de entrada para predecir dos valores de salida (y_1 e y_2). Encontrar un ancho de ventana que resulte en un rendimiento aceptable del modelo no es tarea fácil, por lo que generalmente se deben realizar múltiples pruebas y experimentos.

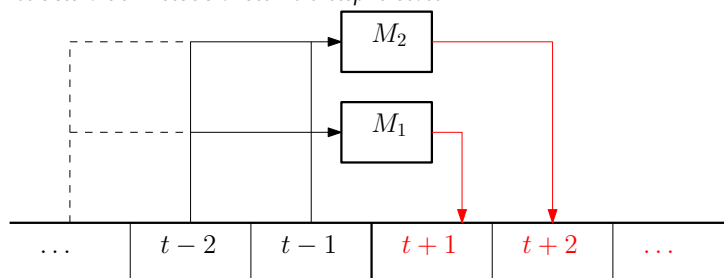
4.1.3. Estrategias de predicción para múltiples pasos

Existen multitud de estrategias para el pronóstico de múltiples pasos (*multi-step time series forecasting*). Una revisión exhaustiva de estos métodos queda fuera del alcance de este texto. Aún así, veremos algunas de las estrategias básicas que pueden emplearse para poder generar predicciones a un número de dos o más instantes de tiempo en el futuro.

Direct multi-step forecast

El método directo (*direct multi-step forecast*) implica desarrollar un modelo separado para cada paso de tiempo de pronóstico. Es decir, si queremos realizar un pronóstico para los siguientes q instantes de tiempo, deberemos desarrollar q modelos independientes, donde cada uno de ellos pronosticará el valor de un solo instante de tiempo $t_i \in \{t+1, \dots, t+q\}$. El esquema de la figura 7 ejemplifica el funcionamiento de esta estrategia en el caso de pronóstico de dos pasos, donde M_1 y M_2 se refiere a dos modelos de aprendizaje automático, que deberán ser entrenados de forma independiente para generar las salidas esperadas para los instantes de tiempo $t+1$ y $t+2$, respectivamente.

Figura 7. Estructura del método *direct multi-step forecast*



Tener un modelo para cada paso de tiempo es una carga adicional de cómputo y mantenimiento, especialmente cuando el número de pasos de tiempo para pronosticar aumenta. Además, en esta estrategia no hay oportunidad de modelar las dependencias entre las predicciones, debido a que se utilizan modelos independientes.

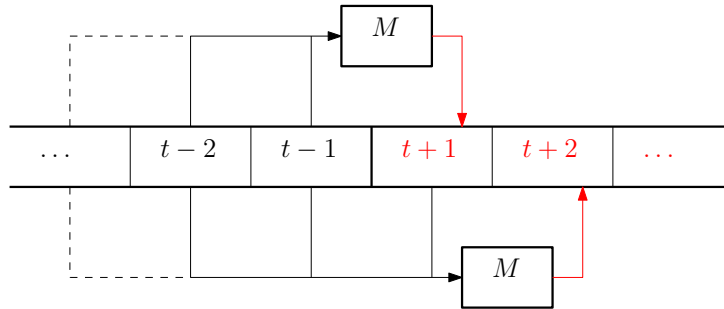
Recursive multi-step forecast

La estrategia recursiva (*recursive multi-step forecast*) implica el uso de un modelo en el que se realiza un paso varias veces, de forma iterativa, utilizando los valores pronosticados en los pasos anteriores para hacer la predicción en el siguiente paso de tiempo.

En esta estrategia, si queremos realizar un pronóstico para los siguientes q

Lectura complementaria

Para más información sobre las estrategias de *multi-step forecast* ver: G. Bontempi; S. Ben Taieb; Y. A. Le Borgne (2013). "Machine Learning Strategies for Time Series Forecasting". En: M. A. Aufaure; E. Zimányi (eds.) *Business Intelligence. eBISS 2012. Lecture Notes in Business Information Processing* (vol. 138). Heidelberg (Berlín): Springer.

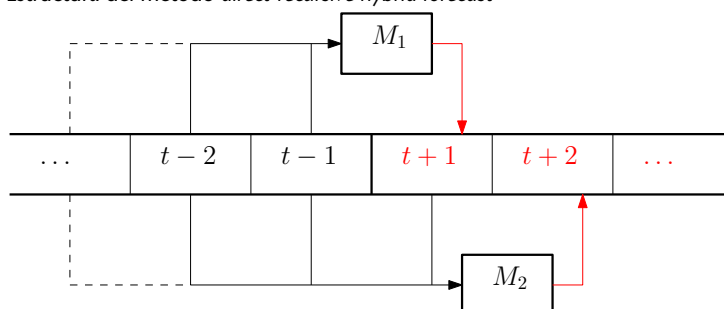
Figura 8. Estructura del método *recursive multi-step forecast*

instantes de tiempo, deberemos aplicar de forma iterativa el mismo modelo q veces, utilizando los valores conocidos de la serie temporal y las últimas predicciones realizadas en los pasos anteriores. La figura 8 muestra el funcionamiento de esta estrategia en el caso de pronóstico de dos pasos, donde M se refiere a un único modelo de aprendizaje automático. M utilizará las entradas $\{t-1, t-2, \dots\}$ para generar el pronóstico de $t+1$ y luego empleará como entradas los valores $\{\text{predicción}(t+1), t-1, t-2, \dots\}$ para generar el pronóstico en $t+2$.

Debido a que las predicciones se usan en lugar de las observaciones, la estrategia recursiva puede acumular errores de predicción de tal manera que el rendimiento pueda degradarse rápidamente a medida que aumenta el horizonte de predicción.

Direct-recursive hybrid strategies

La estrategia híbrida (*direct-recursive hybrid*) combina las estrategias directa y recursiva para ofrecer los beneficios de ambos métodos. En esencia, esta estrategia permite construir un modelo separado para cada paso de tiempo que se debe predecir, pero en este caso cada modelo puede usar las predicciones hechas por modelos en pasos de tiempo anteriores como valores de entrada.

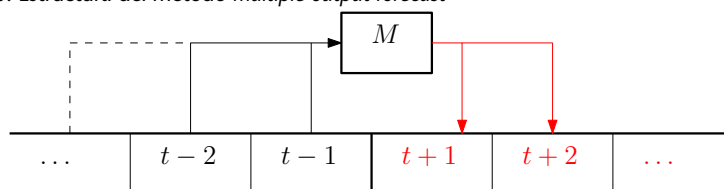
Figura 9. Estructura del método *direct-recursive hybrid forecast*

La figura 9 muestra el funcionamiento de esta estrategia en el caso de pronóstico de dos pasos, donde M_1 y M_2 son dos modelos de aprendizaje automático. El modelo M_1 utilizará las entradas $\{t-1, t-2, \dots\}$ para generar el pronóstico de $t+1$ y, a continuación, el modelo M_2 empleará como entradas los valores $\{\text{predicción}(t+1), t-1, t-2, \dots\}$ para generar el pronóstico en $t+2$.

Multiple output strategy

La estrategia de salida múltiple implica desarrollar un modelo que sea capaz de predecir la secuencia de pronóstico completa de una sola vez. Es decir, el modelo generado deberá tener una salida para cada paso de tiempo que se quiere pronosticar.

Figura 10. Estructura del método *multiple output forecast*



La figura 10 muestra el funcionamiento de la estrategia de salida múltiple en el caso de pronóstico de dos pasos, donde M se refiere al modelo de aprendizaje automático, que utilizará las entradas $\{t-1, t-2, \dots\}$ para generar los pronósticos de $\{t+1, t+2, \dots\}$.

Los modelos de salida múltiple son más complejos, lo que puede significar que son más lentos para entrenar y requieren más datos para evitar sobreajustar el problema (*overfitting*).

Además, es importante notar que este tipo de estrategia solo se puede aplicar en modelos de aprendizaje automático que permiten, explícitamente, definir múltiples salidas del modelo. Por ejemplo, las redes neuronales son un modelo de aprendizaje automático sobre el cual podemos emplear este tipo de estrategia. Por el contrario, las máquinas de vectores de soporte (*support vector machine*, SVM) no permiten múltiples salidas, con lo cual no podemos emplear la estrategia de salida múltiple.

4.2. Métricas para evaluación de series temporales

La evaluación de modelos de series temporales comparte el mismo principio que la evaluación de modelos de regresión, en el sentido de que se comparan los valores predichos con los valores reales de las instancias del conjunto de datos. En este apartado veremos algunas de las métricas más empleadas para evaluar el rendimiento de un modelo de series temporales.

El **error absoluto medio** (*mean absolute error*, MAE) es la métrica más simple y directa para la evaluación del grado de divergencia entre dos conjuntos de valores, representado por la ecuación siguiente:

$$MAE = \frac{1}{|D|} \sum_{d \in D} |f(d) - h(d)|, \quad (4)$$

donde D es el conjunto de instancias, $h : X \rightarrow \mathbb{R}$ es la función del modelo y $f : X \rightarrow \mathbb{R}$ es la función objetivo con las etiquetas correctas de las instancias.

En este caso, todos los residuos tienen la misma contribución al error absoluto final.

El **error cuadrático medio** (*mean square error*, MSE) es, probablemente, la métrica más empleada para la evaluación de modelos de regresión. Se calcula de la siguiente forma:

$$MSE = \frac{1}{|D|} \sum_{d \in D} (f(d) - h(d))^2. \quad (5)$$

Utilizando esta métrica se penalizan los residuos grandes. En este sentido, si el modelo aproxima correctamente gran parte de las instancias del conjunto de datos, pero comete importantes errores en unos pocos, la penalización en esta métrica será muy superior a la indicada si se emplea la métrica anterior (MAE).

La **raíz cuadrada del error cuadrático medio** (*root mean square error*, RMSE) se define aplicando la raíz cuadrada al error cuadrático medio. Tiene las mismas características que este, pero aporta la ventaja adicional de que el error se expresa en la misma escala que los datos originales. En el caso del MSE no era así, ya que la diferencia de valores son elevados al cuadrado antes de realizar la suma ponderada. En algunos casos esta diferencia puede ser importante, mientras que en otros es irrelevante.

$$RMSE = \sqrt{MSE}. \quad (6)$$

El **error porcentual absoluto medio** (*mean absolute percentage error*, MAPE) es una medida de la precisión que se utiliza como una función de pérdida para problemas de regresión en el aprendizaje automático. Por lo general, expresa la precisión como un porcentaje y se define con esta fórmula:

$$MAPE = \frac{100}{n} \sum_{t=1}^n \left| \frac{c_t - y_t}{c_t} \right|, \quad (7)$$

donde c_t y y_t son los valores real y pronosticado, respectivamente, en el instante de tiempo t ; por su parte, n representa el número de predicciones realizadas.

Finalmente, en algunos modelos de regresión se pueden tolerar diferencias importantes entre los valores predichos y verdaderos, siempre que el modelo tenga un comportamiento general similar a los valores verdaderos, prestando especial atención a la monotonicidad. En estos casos, las métricas vistas anteriormente pueden no ser representativas del rendimiento de un modelo de regresión. Por el contrario, los **índices de correlación** lineal o de rango, como por ejemplo Pearson o Spearman, pueden ser una buena métrica para medir la bondad del modelo generado.

5. Ejemplos prácticos

En este último apartado veremos algunos ejemplos prácticos de cómo cargar, procesar y analizar series mediante el lenguaje Python.

En primer lugar, veremos un ejemplo de análisis de series temporales relacionado con la evolución de la concentración de CO₂ en el aire. En segundo lugar, presentaremos un ejemplo relacionado con la predicción de series temporales y nos centraremos en la predicción del nivel de agua de un embalse.

5.1. Ejemplo 1: análisis de la concentración de CO₂

En este primer caso analizaremos cómo evoluciona la concentración de CO₂ en distintos países desde un punto de vista del análisis de series temporales. Es decir, en este ejemplo intentaremos entender el comportamiento de esta serie temporal y los patrones subyacentes, pero nuestro objetivo no será crear un modelo predictivo que nos permita predecir los comportamientos futuros. Los datos empleados en estos ejemplos han sido extraídos del Administración Nacional Oceánica y Atmosférica del departamento de Comercio de los Estados Unidos.

Enlaces de interés

Dr. Pieter Tans de NOAA/ESRL:
<http://www.esrl.noaa.gov/gmd/ccgg/trends/>; Dr. Ralph Keeling de la Institución Scripps de Oceanografía:
<http://scrippsco2.ucsd.edu>.

5.1.1. Carga de los datos

En primer lugar se deben cargar las librerías necesarias para ejecutar correctamente el código de estos ejemplos. La versión de Python empleada es la 3.6.

```
1 import os
2 import pandas as pd
3 import numpy as np
4 import seaborn as sns
5 from matplotlib import pyplot as plt
6 %matplotlib inline
```

A continuación, deberemos cargar los datos. En este caso emplearemos un documento en formato Microsoft Excel obtenido del repositorio de código asociado al libro de Pal y Prakash, donde se pueden encontrar diferentes conjuntos de datos y código en Python para llevar a cabo el análisis.

```
1 data = pd.read_excel('datasets/Monthly_CO2_Concentrations.xlsx', converters={'Year': np.int32,
    'Month': np.int32})
2
3 data = data.loc[(~pd.isnull(data['CO2'])) & \
4 (~pd.isnull(data['Year'])) & \
5 (~pd.isnull(data['Month']))]
6
7 data.sort_values(['Year', 'Month'], inplace=True)
```

Lectura complementaria

Este ejemplo está basado en Avishek Pal; P. K. S. Prakash (2017). *Practical Time Series Analysis*. Packt Publishing.

Enlace de interés

El libro de Pal y Prakash está disponible en <https://github.com/PacktPublishing/Practical-Time-Series-Analysis>.

En el código anterior se cargan los datos del fichero y, posteriormente, se eliminan las filas que contengan valores ausentes (el valor `NULL`) y se ordenan según el valor de los campos año (`Year`) y mes (`Month`).

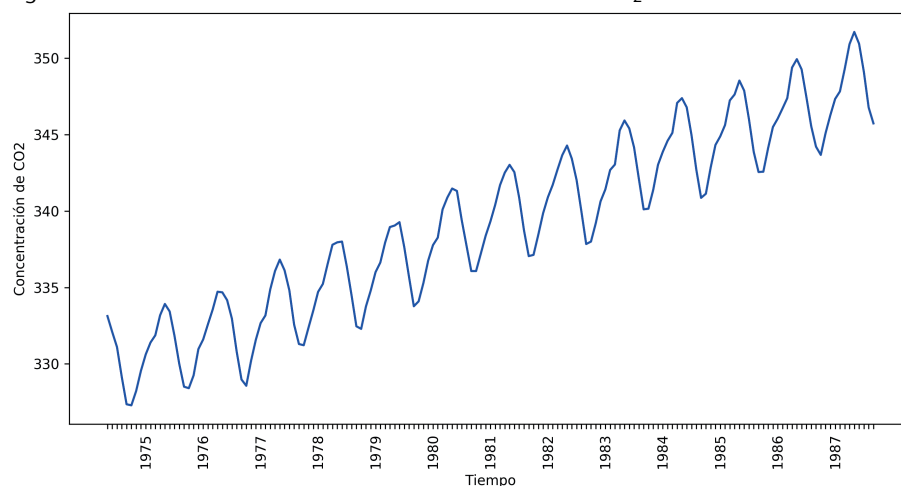
5.1.2. Tendencia general

A continuación creamos una gráfica para poder analizar de forma visual el comportamiento de los datos de la serie temporal. El resultado se puede ver en la figura 11, donde observamos que hay una tendencia al alza de los datos sobre la concentración de CO_2 .

```
1 plt.figure(num=1, figsize=(11, 5.5))
2
3 plt.plot(data['CO2'], color='b')
4 plt.xlabel('Tiempo')
5 plt.ylabel('Concentración de CO2')
6 ax = plt.gca()
7 ax.set_xticklabels([])
8
9 plt.show()
```

Como hemos visto anteriormente en este módulo, la tendencia general es fácil de identificar de forma visual si se dispone de un conjunto suficientemente grande como para identificar la tendencia a largo plazo, pero cuando no se dispone de un volumen de datos suficiente es difícil de identificar.

Figura 11. Visualización de los datos sobre la concentración de CO_2



La tendencia general se puede modelar, entre otras formas, empleando un modelo de regresión lineal que nos indique la pendiente de inclinación de los datos a largo plazo. Además, la línea de regresión o tendencia se puede usar como una predicción del movimiento a largo plazo de las series temporales.

El siguiente fragmento de código crea un modelo de regresión lineal para los datos de la serie temporal y, a continuación, calcula el valor de predicción para cada punto del conjunto de datos. Es decir, calculamos el valor de la predicción para cada uno de los puntos de tiempo de la serie temporal.


```

1 from sklearn.linear_model import LinearRegression
2
3 trend_model = LinearRegression(normalize=True, fit_intercept=True)
4 trend_model.fit(np.arange(data.shape[0]).reshape((-1,1)), data['CO2'])
5
6 print('Trend model coefficient={} and intercept={}'.format(trend_model.coef_[0],trend_model.
    intercept_))
7
8 predictions = trend_model.predict(np.arange(data.shape[0]).reshape((-1,1)))

```

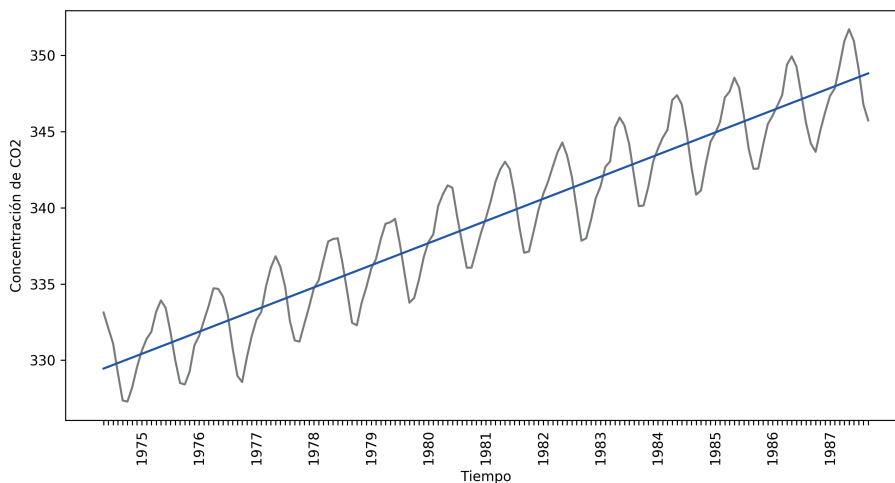
Una vez tenemos los valores de la regresión, podemos generar una gráfica que nos permite ver la aproximación que ha generado la recta de regresión respecto a los datos originales. El código que se muestra a continuación genera dicha gráfica, que podemos ver en la figura 12. Como se puede observar, en este caso una recta de regresión permite ajustar bastante bien la tendencia general de los datos originales.

```

1 plt.figure(num=1, figsize=(11, 5.5))
2
3 plt.plot(data['CO2'], color='gray')
4 plt.plot(predictions, color='b')
5 plt.xlabel('Tiempo')
6 plt.ylabel('Concentración de CO2')
7 ax = plt.gca()
8 ax.set_xticklabels([])
9
10 plt.show()

```

Figura 12. Tendencia general de los datos a partir de una recta de regresión



A partir de los datos de tendencia general podemos obtener los **datos residuales**, que se definen como los datos obtenidos a partir de sustraer la tendencia general a los datos originales, tal y como podemos ver en el siguiente fragmento de código.

```

1 residuals = data['CO2'] - predictions

```

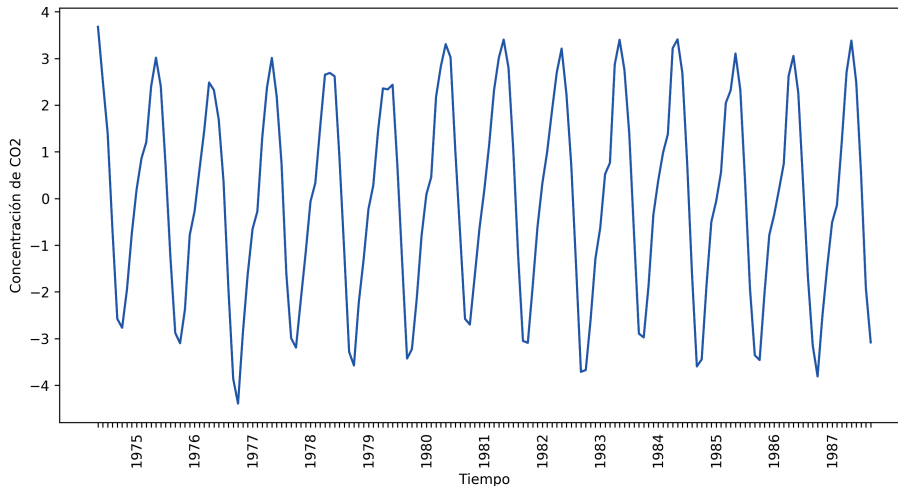
Los residuos obtenidos al sustraer la línea de tendencia de los datos originales se analizan en busca de otras propiedades interesantes, como la estacionalidad, el comportamiento cíclico y las variaciones irregulares. La figura 13 muestra los datos residuales.

```

1 plt.figure(figsize=(11, 5.5))
2
3 plt.plot(residuals, color='b')
4 plt.xlabel('Tiempo')
5 plt.ylabel('Concentración de CO2')
6 ax = plt.gca()
7 ax.set_xticklabels(labels)
8 plt.xticks(rotation=90)
9
10 plt.show()

```

Figura 13. Residuos generados a partir de sustraer la tendencia general a los datos originales



5.1.3. Movimientos estacionales

Como se ha comentado anteriormente, podemos identificar las variaciones estacionales a partir de los patrones observados en una visualización que comprenda varios años completos y que, por lo tanto, nos permita identificar patrones subyacentes que respondan a movimientos estacionales.

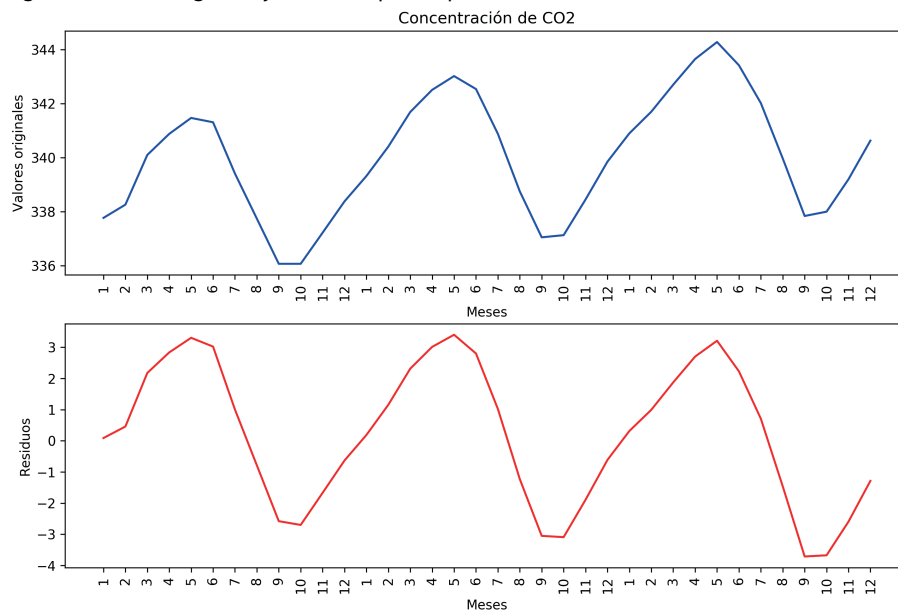
En el siguiente código se crea una gráfica que muestra los datos filtrados entre los años 1980 y 1982. El resultado se puede ver en la figura 14. Por un lado, se muestran los datos originales para este período de tiempo (figura superior); por otro lado, se muestran los datos residuales para el mismo período de tiempo (figura inferior).

```

1 # filtramos por los años 1980-1982
2 data_tmp = data.loc[(data['Year']==1980) or (data['Year']==1981) or (data['Year']==1982)]
3
4 plt.figure(figsize=(11.5, 7.5))
5
6 plt.subplot(211)
7 plt.plot(data_tmp['CO2'], color='b')
8 plt.title('Concentración de CO2')
9 plt.xlabel('Meses')
10 plt.ylabel('Valores originales')
11 ax = plt.gca()
12 ax.set_xticklabels(data_tmp['Month'])
13 plt.xticks(rotation=90)
14
15 plt.subplot(212)
16 plt.plot(data_tmp['Residuals'], color='r')
17 plt.xlabel('Meses')
18 plt.ylabel('Residuos')
19 ax = plt.gca()
20 ax.set_xticklabels(data_tmp['Month'])
21 plt.xticks(rotation=90)
22
23 plt.show()

```

Figura 14. Datos originales y residuales para el período 1980-1982



Otra forma de determinar la variación estacional de los datos es agregando los datos individuales en períodos de mayor envergadura y observando cómo se comportan valores agregados sobre estos datos, como por ejemplo el valor promedio y la desviación estándar. En el código siguiente creamos una función (`month_quarter_map`) que asigna el trimestre correspondiente a cada fila de datos en una nueva columna (`Quarter`).

```
1 # Creamos la función de mapeo: mes => período
2 month_quarter_map = {1: 'Q1', 2: 'Q1', 3: 'Q1',
3   4: 'Q2', 5: 'Q2', 6: 'Q2',
4   7: 'Q3', 8: 'Q3', 9: 'Q3',
5   10: 'Q4', 11: 'Q4', 12: 'Q4'}
6
7 data['Quarter'] = data['Month'].map(lambda m: month_quarter_map.get(m))
```

Seguidamente creamos los datos agregados por trimestre y calculamos el valor medio y la desviación estándar para cada grupo. El primer grupo estará formado por los datos de los meses de enero, febrero y marzo; el segundo por los datos de abril, mayo y junio, y así sucesivamente hasta crear cuatro grupos para cubrir los doce meses del año.

```
1 seasonal_sub_series_data = data.groupby(by=['Year', 'Quarter'])['Residuals'].aggregate([np.
2   mean, np.std])
3
4 # Creamos índices de fila para seasonal_sub_series_data utilizando el valor del año y
5   trimestre
6 seasonal_sub_series_data.reset_index(inplace=True)
7 seasonal_sub_series_data.index = seasonal_sub_series_data['Year'].astype(str) + '-' +
8   seasonal_sub_series_data['Quarter']
9 seasonal_sub_series_data.head()
```

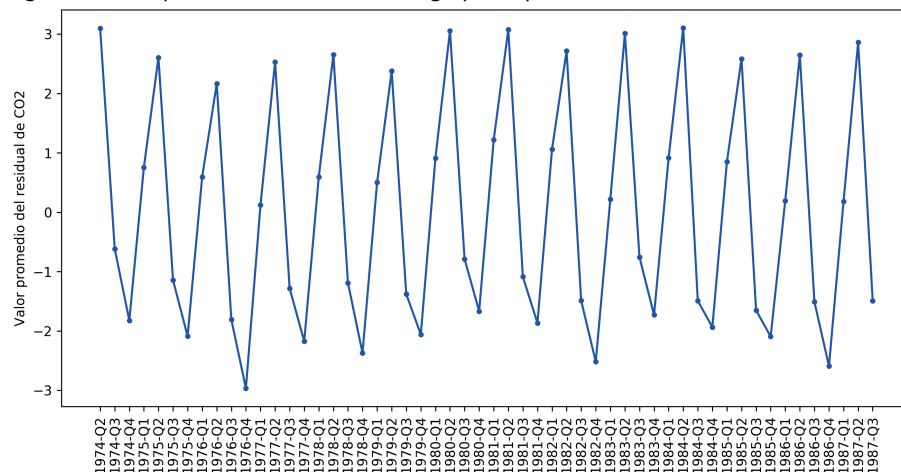
El siguiente fragmento de código genera el gráfico que mostrará el valor promedio de los residuos según los valores agregados por trimestre que acabamos de generar. La figura 15 presenta la visualización resultante, donde podemos ver que el valor promedio depende en gran medida del trimestre. También se muestran valores muy parecidos en los residuos de todos los años incluidos en los datos originales.

```

1 plt.figure(figsize=(11.5, 5.5))
2
3 plt.plot(seasonal_sub_series_data['Quarterly Mean'], color='b', marker='.')
4 plt.xlabel('Trimestres')
5 plt.ylabel('Valor promedio del residual de CO2')
6 plt.xticks(rotation=90)
7
8 plt.show()

```

Figura 15. Valor promedio de los residuales agrupados por trimestre



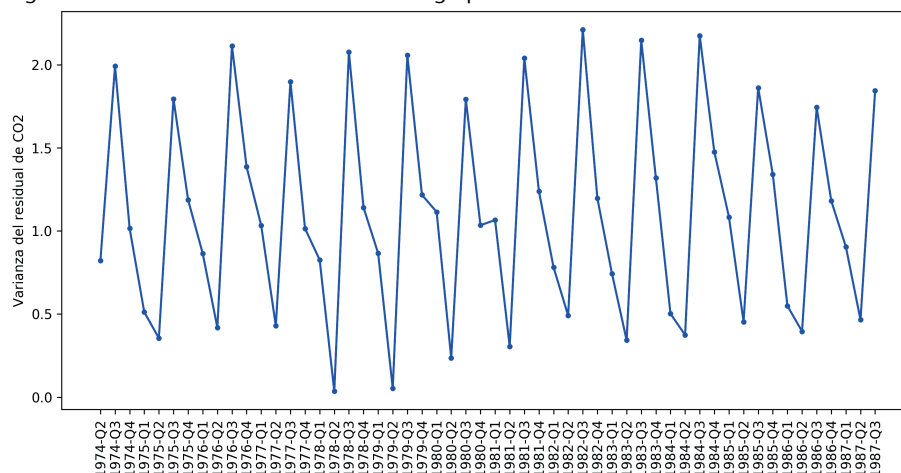
De forma similar, el siguiente fragmento de código genera el gráfico que mostrará la varianza de los residuos según los valores agregados por trimestre que acabamos de generar. El resultado se puede ver en la figura 16, donde podemos ver que la varianza depende claramente del trimestre. Se muestran valores muy parecidos en los residuos de todo el conjunto de datos.

```

1 plt.figure(figsize=(11.5, 5.5))
2
3 plt.plot(seasonal_sub_series_data['Quarterly Standard Deviation'], color='b', marker='.')
4 plt.xlabel('Tiempo')
5 plt.ylabel('Varianza del residual de CO2')
6 plt.xticks(rotation=90)
7
8 plt.show()

```

Figura 16. Varianza de los valores residuales agrupados



Finalmente, podemos emplear un diagrama de cajas para ver, de forma unificada, los valores promedio y varianza para una agregación según trimestre de todos los años incluidos en el análisis. El siguiente código muestra cómo generar este gráfico empleando la función `boxplot` de la librería `seaborn` de Python.

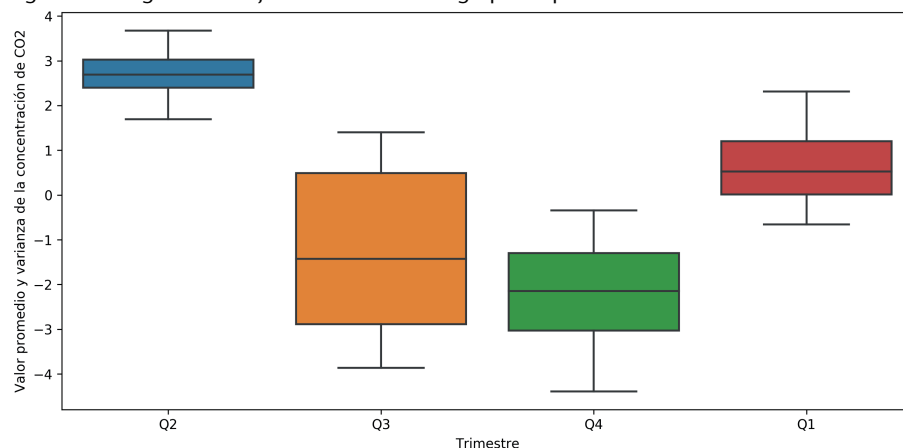
```

1 plt.figure(figsize=(11.5, 5.5))
2
3 g = sns.boxplot(data=data, y='Residuals', x='Quarter')
4 g.set_xlabel('Trimestre')
5 g.set_ylabel('Valor promedio y varianza de la concentración de CO2')
6
7 plt.show()

```

El resultado puede verse en la figura 17. Como se puede observar, los valores promedio y varianza tienen una clara dependencia con el trimestre, independientemente del año.

Figura 17. Diagrama de cajas de los residuales agrupados por trimestre



5.1.4. Estacionariedad de la serie temporal

Finalmente, y para terminar con este ejemplo, veremos si la serie temporal que estamos analizando es estacionaria. Aunque las gráficas vistas en las secciones anteriores ya nos indican que la serie temporal no es estacionaria, aplicaremos el test ADF para comprobar, de forma numérica, el resultado de esta prueba que nos indica si una serie temporal es estacionaria.

El siguiente fragmento de código carga la función `adfuller` de la librería `statsmodels`, que nos permite realizar la prueba.

```

1 from statsmodels.tsa.stattools import adfuller
2
3 result = adfuller(data['CO2'])
4
5 print('ADF Statistic: %f' % result[0])
6 print('p-value: %f' % result[1])
7 print('Critical Values:')
8 for key, value in result[4].items():
9     print('\t%s: %.3f' % (key, value))

```

La salida que nos reporta el test ADF es la siguiente:

```

ADF Statistic: -0.355668
p-value: 0.917242
Critical Values:
1%: -3.476
5%: -2.882
10%: -2.577

```

Como vimos anteriormente, un valor de $p > 0,05$ indica que no se puede rechazar la hipótesis nula (H_0), por lo tanto, los datos tienen una raíz unitaria y no son estacionarios. Además, el valor estadístico de prueba es -0.35 , lo que indica una muy baja probabilidad de rechazar la hipótesis nula.

En resumen, podemos concluir que la serie temporal de concentración de CO_2 no es estacionaria.

5.2. Ejemplo 2: predicción del nivel de un embalse

En el segundo ejemplo analizaremos cómo evoluciona el nivel de agua de un embalse desde el punto de vista de la predicción de series temporales. Es decir, nuestro objetivo principal será crear un modelo predictivo que nos permita predecir cómo evolucionará el nivel del agua en el embalse en el futuro.

Los datos empleados en este ejemplo han sido extraídos de la Agència Catalana de l'Aigua (ACA). En concreto, se han obtenido los datos sobre el embalse de Sau, ubicado en Cataluña, en el período comprendido entre los años 2007 y 2019.

5.2.1. Adquisición de datos

En primer lugar, cabe notar que los datos originales obtenidos de la ACA están en formato XML, con lo cual el primer paso es convertir este tipo de documento en un formato de tabla, donde cada fila representa una observación y cada columna es un atributo. En este caso concreto hemos obtenido observaciones con una frecuencia diaria y cinco atributos distintos del embalse. Los atributos se presentan en la tabla 9:

Tabla 9. Atributos disponibles del embalse de Sau

Variable	Descripción	Unidades
4165141	Entrada	m^3/s
4165140	Salida	m^3/s
4159510	Volumen	hm^3
4159547	Capacidad	%
4159509	Nivel	m s. n. m.
3378678	Grupos 1 + 2	m^3/s

Como se puede ver en la tabla, los datos obtenidos proporcionan información sobre el volumen de entrada y salida del embalse (hm^3/s), el volumen total de agua (hm^3), el porcentaje de capacidad del embalse y el nivel de la superficie del agua del embalse (metros sobre el nivel del mar). En este caso, hemos escogido la variable **volumen** (hm^3) como valor de referencia para el análisis.

Enlace de interés

Los datos sobre el embalse de Sau se han extraído de https://es.wikipedia.org/wiki/Embalse_de_Sau.

Lecturas complementarias

R. Parada; J. Font; J. Casas-Roma (2019). *Predicting Energy Generation Using Forecasting Techniques in Catalan Reservoirs*. *Energies* (núm. 12, pág. 1832). <https://doi.org/10.3390/en12101832>

```
1 import pandas as pd
2 import numpy as np
3
4 data = pd.read_csv('data_sau.csv', sep=';', header=0, index_col=0)
5 dataframe = pd.DataFrame(data.loc[data.index >= '2007-01-01']['Volum'])
6 dataset = dataframe.values
7 dataset = dataset.astype('float32')
```

En el código anterior vemos cómo se carga el fichero CSV con los datos pre-procesados y se filtran los datos anteriores al 1 de enero de 2007. Además, se selecciona únicamente el atributo o la variable volumen (`volum`), que es el dato que hemos seleccionado para analizar en este ejemplo.

5.2.2. Análisi preliminar

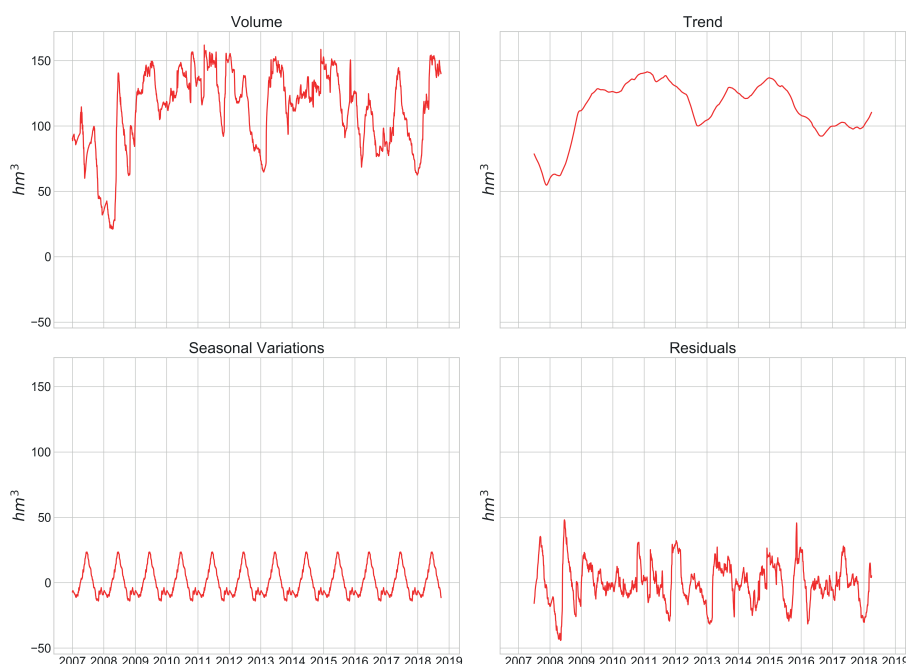
A partir de los datos tratados en el apartado anterior, realizaremos un primer análisis de la tendencia, variaciones estacionales, etc. Este tipo de análisis es muy útil para entender los datos con los que se está trabajando.

El siguiente código muestra cómo realizar la descomposición de la serie temporal empleando la librería `statsmodels` de Python. El resultado obtenido incluye cuatro subconjuntos de datos: los datos originales (`observed`), la tendencia general (`trend`), las variaciones estacionales (`seasonal`) y los residuos (`resid`).

```
1 import statsmodels.api as sm
2
3 # time serie decomposition
4 descomp_serie = sm.tsa.seasonal_decompose(dataset, model='additive', freq=365)
5
6 print(descomp_serie.observed)
7 print(descomp_serie.trend)
8 print(descomp_serie.seasonal)
9 print(descomp_serie.resid)
```

En la figura 18 podemos ver los valores originales y los tres componentes obtenidos en el proceso de descomposición de la serie temporal. Se puede apreciar una clara variación estacional, producida sin duda por los períodos de lluvia abundante en esta región del sur de Europa. También es importante notar la variación del conjunto original de datos, con un valor mínimo de 21 hm^3 , un valor máximo de 162 hm^3 y un valor medio de 112 hm^3 .

Figura 18. Descomposición de la serie temporal.



En este caso, la salida que nos reporta el test ADF es la siguiente:

```
ADF Statistic: -3.759525
p-value: 0.003350
Critical Values:
1%: -3.431880
5%: -2.862216
10%: -2.567130
```

Como vimos anteriormente, un valor de $p < 0,05$ indica que se puede rechazar la hipótesis nula (H_0) y, por lo tanto, los datos son estacionarios.

5.2.3. Preparación de los datos

En este ejemplo tratamos con un problema univariante, ya que solo hemos seleccionado una de las variables o atributos disponibles. Pero debemos seleccionar dos parámetros muy importantes:

- El número de días previos que emplearemos para el modelo predictivo.
- El número de pasos al que deseamos formular las predicciones.

Para este ejemplo supondremos que deseamos realizar una predicción a quince días. Es decir, queremos que el modelo de aprendizaje automático nos muestre los valores futuros para los próximos quince días. Este parámetro se define a partir de los requerimientos del problema, pero lógicamente se debe tener en cuenta que las predicciones a largo plazo suelen tener un error mucho mayor que las predicciones a corto plazo.

En relación con el número de días previos que empleará el modelo para generar la predicción, este parámetro generalmente involucra algunas pruebas para determinar el valor óptimo o, por lo menos, un valor aceptable. En nuestro caso, después de realizar algunas pruebas hemos decidido fijarlo en quince días, como en el caso anterior.

En resumen, el modelo empleará los valores reales del volumen de agua de los últimos quince días para generar una predicción del volumen para los próximos quince días. El conjunto de datos deberá tener la estructura que se muestra en la tabla 10:

Tabla 10. Estructura del conjunto de datos preparado para el modelo predictivo

$X_{(t-15)}$	$X_{(t-14)}$	...	$X_{(t-1)}$	$Y_{(t+1)}$	$Y_{(t+2)}$	...	$Y_{(t+15)}$
112.87	113.15	...	114.10	115.09	118.14	...	113.76

Para generar esta estructura a partir de los datos originales, emplearemos el método de la ventana deslizante (*sliding window method*) con un ancho de ventana (*window width*) igual a 15. El siguiente código permite generar el conjunto de datos necesarios para entrenar el modelo de aprendizaje automático.


```

1 from sklearn.model_selection import train_test_split
2 from numpy import array
3
4 # split a univariate sequence into samples
5 def split_sequence(sequence, n_steps_in, n_steps_out):
6     X, y = list(), list()
7
8     for i in range(len(sequence)):
9         # find the end of this pattern
10        end_ix = i + n_steps_in
11        out_end_ix = end_ix + n_steps_out
12        # check if we are beyond the sequence
13        if out_end_ix > len(sequence):
14            break
15        # gather input and output parts of the pattern
16        seq_x, seq_y = sequence[i:end_ix], sequence[end_ix:out_end_ix]
17        X.append(seq_x)
18        y.append(seq_y)
19
20    return array(X), array(y)
21
22 # params
23 n_steps_in = 15
24 n_steps_out = 15
25
26 # create sequence
27 X, y = split_sequence(dataset, n_steps_in, n_steps_out)
28
29 # create train and test datasets
30 train_X, test_X, train_Y, test_Y = train_test_split(X, y, test_size=0.33)

```

La salida de este fragmento de código son las cuatro variables estándar para el entrenamiento de un modelo predictivo:

- `train_X`: variable con los datos de entrada del modelo (x_i). En este caso concreto, tendremos quince valores para cada fila, dado que hemos determinado utilizar los quince días anteriores para realizar los pronósticos.
- `test_X`: variables con los datos de entrada del modelo, pero reservados para la fase de test.
- `train_Y`: variables con los datos de salida del modelo (y_i). En este caso, también tendremos quince valores, ya que queremos que el modelo haga predicciones en un futuro de quince días.
- `test_Y`: variables con los datos de salida del modelo, pero reservados para la fase de test.

5.2.4. Creación del modelo predictivo

Una vez tenemos el conjunto de datos preparado, podemos empezar con la creación del modelo predictivo y su entrenamiento. Existen una gran diversidad de modelos de aprendizaje automático, cada uno de ellos con ciertas características, virtudes y defectos. Queda fuera del alcance de este texto una revisión de estos modelos.

En este ejemplo emplearemos las máquinas de vectores soporte (*support vector machines*, SVM). Las SVM son uno de los modelos más empleados en aprendizaje automático y, concretamente, para resolver problemas de regresión, como el que planteamos. Las SVM no permiten múltiples salidas, por lo que no es posible emplear la estrategia MIMO para generar la predicción a quince días. En este caso emplearemos la estrategia directa.

```

1 from sklearn.svm import SVR
2 from sklearn.multioutput import MultiOutputRegressor
3
4 model = MultiOutputRegressor(SVR(kernel='rbf', C=20, gamma=0.28, epsilon=0.0125))
5 # train the model
6 model.fit(train_X, train_Y)
7
8 # test the model (new data)
9 pred_Y = model.predict(test_X)

```

En el código anterior, se ha empleado la función `MultiOutputRegressor` de la librería `sklearn`, que permite aplicar el modelo directo para generar múltiples salidas en modelos de aprendizaje automático que no soportan salidas múltiples. La propia librería se encarga de entrenar los diferentes modelos, uno para cada atributo o variable de salida.

5.2.5. Resultados

Figura 19. Evolución del RMSE en función del número de días

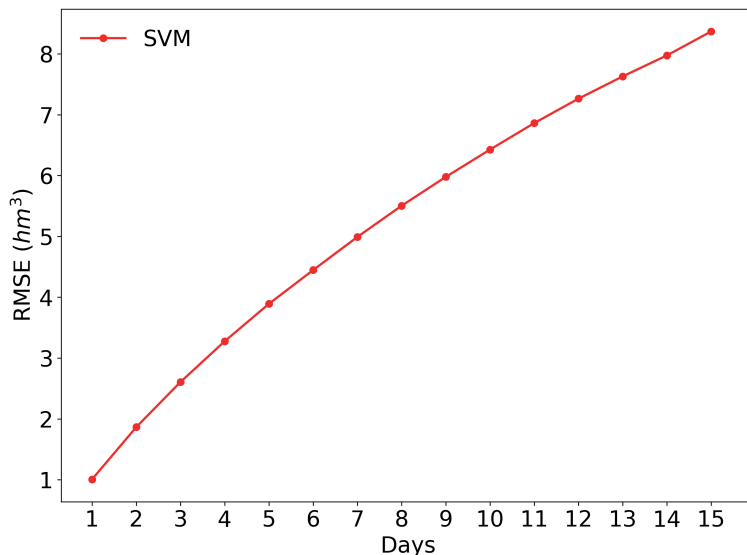
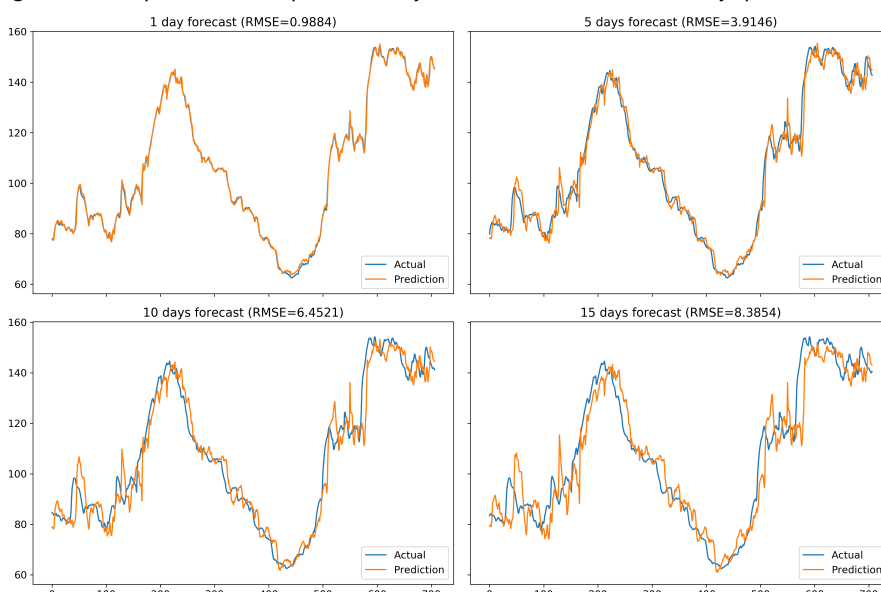


Figura 20. Comparación de las predicciones y datos reales a uno, cinco, diez y quince días



A continuación, podemos ver el error que ha cometido el modelo en la predicción de valores futuros. La figura 19 muestra cómo evoluciona la raíz cuadrada del error cuadrático medio (*root mean square error*, RMSE) según el número de días que empleamos en la predicción. Lógicamente, cuanto mayor es el número de días, mayor es el error que comete el modelo.

Finalmente, la figura 20 presenta la comparación entre los resultados reales (*actual*) y predichos (*prediction*) para un subconjunto de dos años, presentando los resultados en función del pronóstico a uno, cinco, diez y quince días. Se puede ver claramente que las predicciones a quince días tienen un grado de divergencia respecto a los datos reales mucho mayor que en los casos de uno y cinco días.

Resumen

En este módulo hemos visto una introducción al análisis de series temporales. En primer lugar, hemos definido qué es una serie temporal, hemos descrito sus principales características y objetivos, y hemos presentado las tareas analíticas asociadas a este tipo de datos. También se han definido los cuatro componentes principales que forman una serie temporal y que permiten entender el comportamiento y los patrones subyacentes en las series temporales. En concreto, los componentes que hemos discutido son: variaciones a largo plazo o tendencia general; variaciones a corto plazo, que pueden ser estacionales o cíclicas, y variaciones aleatorias o irregulares. Finalmente, hemos definido el concepto de *estacionaridad* en las series temporales y presentado algunos métodos básicos para determinar si una serie es estacionaria y, en caso de no serlo, cómo se puede transformar en una serie estacionaria.

A continuación hemos revisado algunas técnicas básicas para transformar y reestructurar los datos de series temporales para que puedan ser empleados en algoritmos estándares de aprendizaje automático (*machine learning*). Hemos definido los conceptos de *one-step* y *multi-step forecast* y hemos visto que la técnica de la ventana deslizante nos permite adaptar los datos para generar conjuntos de entrenamiento en modelos de aprendizaje automático, así como algunas técnicas básicas para poder desarrollar modelos predictivos de múltiples pasos (*multi-step forecast*). Seguidamente, hemos revisado algunas de las métricas más empleadas para la evaluación de la bondad en la predicción de series temporales.

Hemos finalizado este módulo con dos pequeños ejemplos prácticos, en los que hemos aplicado, empleando el lenguaje de programación Python y algunas de sus librerías más interesantes y populares, algunos de los conocimientos adquiridos a lo largo de este texto para el análisis y la predicción de series temporales.

Glosario

indicadores del desarrollo mundial *m* es la principal compilación de estadísticas internacionales sobre desarrollo global del Banco Mundial. Utilizando fuentes reconocidas oficialmente e incluyendo estimaciones nacionales, regionales y mundiales, el WDI proporciona acceso a aproximadamente 1.600 indicadores para 217 economías, con algunas series temporales que se extienden más de 50 años.

sigla **WDI**

en world development indicators

internet de las cosas *m* Concepto que se refiere a la interconexión digital de objetos cotidianos con internet.

sigla **IoT**

en Internet of Things

producto interior bruto *m* Magnitud macroeconómica que expresa el valor monetario de la producción de bienes y servicios de demanda final de un país o región durante un período determinado, normalmente de un año.

sigla **PIB**

taxonomía *f* Clasificación u ordenación en grupos de cosas que tienen unas características comunes.

Bibliografía

Bontempi G.; Ben Taieb S.; Le Borgne Y. A. (2013). "Machine Learning Strategies for Time Series Forecasting". En: M. A. Aufaure; E. Zimányi (eds.) *Business Intelligence. eBISS 2012. Lecture Notes in Business Information Processing*. (vol. 138). Heidelberg (Berlín): Springer.

Lazzeri, F. (2019). *Time Series Forecasting*. O'Reilly Media, Inc.

Nielsen, A. (2019). *Practical Time Series Analysis*. O'Reilly Media, Inc.

Pal, A.; Prakash, P. K. S. (2017). *Practical Time Series Analysis*. Packt Publishing.